Sentinel-1 Oil Spill Project

Advanced Viva & Engineering Questions — Simple Explanations

This guide adds the higher-level questions that can come after the basic ML, U-Net, evaluation, and geospatial theory: cost, data sourcing, scalability, environmental limitations, commercial use, tensor computation, deterministic post-processing, and geospatial vectorization.

1. Cost & Data Sourcing

Likely judge question: “How much does it cost to run this if we scale it?”

> **Core answer:** Sentinel-1 data is freely accessible through the Copernicus Data Space Ecosystem. Therefore, the major operational costs when scaling the system are compute, storage, networking, and related cloud infrastructure rather than purchasing the satellite imagery.

Important wording: Say “free/open-access satellite data” rather than “open-source satellite data.” Open-source normally describes software or source code.

2. Scalability & Cloud Architecture

Likely judge question: “You trained this on an RTX 4060. How would you scan the entire Indian coastline?”

Current prototype: The model and processing pipeline can run locally on the RTX 4060 used during development.

> **Scalable design:** The system can be migrated to a cloud architecture in which the React/Mapbox dashboard is deployed separately from the PyTorch inference engine. A GPU-enabled server or instance can perform model inference, while an API connects the inference service with the web application.

Conceptual architecture:

Layer	Role
Sentinel-1 data	Source SAR imagery
Preprocessing	Read, normalize, and prepare imagery
GPU inference service	Run the PyTorch U-Net model
Post-processing	Clean predicted regions and vectorize them
API / backend	Transfer results to the dashboard
React + Mapbox	Display detections geographically

Do not overclaim: If you have not actually implemented microservices/cloud deployment, describe this as a proposed scalable architecture rather than saying you already built it.

3. Environmental Edge Cases: Extreme Weather

Likely judge question: “What happens during a massive hurricane or severe storm?”

> **Core answer:** SAR observes microwave backscatter from the sea surface rather than directly observing oil. Strong winds and rough seas can significantly change the radar backscatter and make oil signatures harder to distinguish from natural sea-surface patterns.

As a result, extreme weather can increase both false positives and false negatives. This is an important limitation because the model's performance depends not only on the neural network but also on the physical conditions under which the SAR image was acquired.

Viva takeaway: A mature system should acknowledge SAR's physical limitations instead of presenting the model as infallible.

4. Commercial / Operational Viability

Likely judge question: “Who would actually use this?”

> Potential users include maritime authorities, environmental agencies, and organizations involved in oil-spill monitoring. The proposed benefit is to reduce manual image inspection and help prioritize areas for physical investigation.

Important distinction: The system should be presented as a decision-support or monitoring tool, not as an automatic replacement for Coast Guard or environmental-response operations. A useful workflow is: AI detects candidate regions → system prioritizes them → human/authority verifies and acts.

5. Language & Problem-Solving Theory

Primary programming language: Python.

However, the project is not just “a Python program.” It combines machine learning, numerical tensor computation, image processing, and geospatial data processing.

5.1 Tensor Algebra & Differentiable Programming

Simple idea: The U-Net does not literally “look” at an image like a human. The image is represented numerically as tensors, and the neural network performs many numerical operations—especially convolutions and tensor/matrix operations—to transform the input into predictions.

> For this project, the input contains two channels, VV and VH, which can be viewed as a numerical array with dimensions related to height, width, and channels. During training, the model adjusts its parameters to reduce the chosen loss function, such as Dice Loss.

Important correction: Do not say the entire method is simply “probabilistic.” The network produces class scores/probabilities, while training is a numerical optimization process driven by the loss and gradients.

5.2 Heuristic Filtering — Deterministic Logic

Simple idea: Your pipeline is not purely AI. After the U-Net produces a prediction, OpenCV performs a deterministic post-processing step.

> Connected Component Analysis identifies contiguous predicted regions. Your pipeline then applies a fixed 50-pixel threshold: components at or below that threshold can be removed as very small regions.

Why this matters: The neural network handles fuzzy pattern recognition, while the post-processing rule applies predictable geometric logic. Together they form a hybrid ML + traditional image-processing pipeline.

If asked “Why exactly 50?”: Do not invent a mathematical justification unless your experiments established one. State that it is a post-processing threshold intended to remove very small predicted components and that its value can be tuned experimentally.

5.3 Geospatial Vectorization

Simple idea: The neural network works in image-grid coordinates, but the final result needs to refer to real places on Earth.

> A predicted pixel has an image position such as (x, y). Rasterio uses the raster's affine transform and coordinate reference system (CRS) to map those grid positions to geographic coordinates.

The pipeline can then convert the cleaned raster mask into polygons using raster-to-vector conversion and output them as GeoJSON. EPSG:4326 provides a standard geographic coordinate system for representing longitude/latitude coordinates.

6. Full Engineering Mental Model

Stage	What happens
1. Sentinel-1 SAR	Obtain VV/VH satellite imagery
2. Raster preprocessing	Read and normalize TIFF data
3. Tensor conversion	Represent image data numerically for PyTorch
4. U-Net + ResNet34	Predict the class of each pixel
5. Post-processing	Remove very small connected components
6. Vectorization	Convert the cleaned raster mask into polygons
7. Geospatial transform	Map pixel positions to geographic coordinates

8. GeoJSON	Package detections as geographic features
9. Dashboard	Display results on a map

One-sentence project explanation: “We use Sentinel-1 SAR imagery as a two-channel input to a U-Net segmentation model, clean the resulting predictions with deterministic image processing, and convert the detections into geographically referenced GeoJSON polygons for visualization and further investigation.”

7. Questions You Should Be Ready For

• Why is Sentinel-1 useful for this problem?

• Is the satellite data free, and what actually costs money?

• How would you move from an RTX 4060 prototype to cloud-scale inference?

• What happens when weather conditions make SAR signatures unreliable?

• Why is the model a decision-support tool rather than a replacement for field verification?

• What exactly is a tensor in your project?

• Why does the model need VV and VH as two input channels?

• What does Connected Component Analysis do?

• Why is the 50-pixel threshold deterministic?

• How does a pixel prediction become a real geographic polygon?

• What are an affine transform and CRS?

• Why output GeoJSON instead of keeping only the segmentation mask?

• Which parts are AI and which parts are traditional programming?

• What would you change to make the system scalable?

Viva strategy: Be precise about what you actually implemented versus what is a proposed future architecture. Judges generally get more useful information from a clear statement of limitations and design trade-offs than from claims that the prototype already solves every deployment problem.